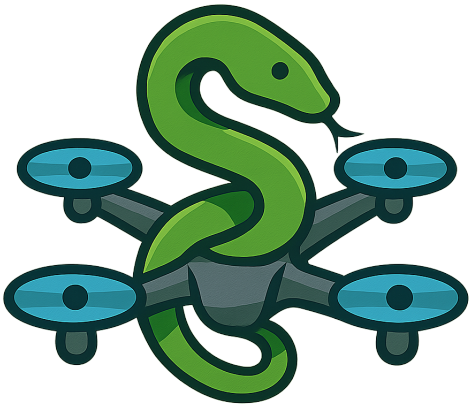
Technische Hochschule Ostwestfalen-Lippe

Bachelor of Science – Data Science

Software Design



Ausarbeitung zum Thema:

Python with Drones

Erstellt von:	Niclas Falke, Stefan Reitemeyer und Jonas Pottmeier
Matrikelnummer:	20122542 (Niclas), 20124913 (Stefan) und 20116026 (Jonas)
Fachsemester:	3
Abgabedatum:	08.04.2026
Prüfer:	Prof. Dr.-Ing Rainer Rasche

Inhaltsverzeichnis

1 Abstrakt **1**

2 Einleitung **1**

3 Grundlagen **1**

3.1 Spielbasiertes Lernen und Computational Thinking 1

3.2 Browserbasierte Python-Ausführung 2

3.3 Verwendete Technologien 2

3.3.1 Next.js und TypeScript 2

3.3.2 Three.js 3

3.3.3 Pyodide 3

3.3.4 shadcn 4

3.3.5 GitHub Actions und Pages 4

4 Implementierung **4**

4.1 Systemarchitektur & Komponentenübersicht 4

4.2 Level-System & Datenstruktur 6

4.3 Python-Ausführung im Browser 6

4.4 Kommunikation zwischen Python und JavaScript 7

4.5 Visualisierung der 3D-Szene 8

5 Ergebnisse **8**

6 Auswertung & Diskussion **9**

7 Schlussfolgerung **9**

7.1 Zusammenfassung 9

7.2 Nächste Schritte und Erweiterungsmöglichkeiten 10

8 Literaturverzeichnis **11**

1 Abstrakt

Die vorliegende Arbeit befasst sich mit der Entwicklung von *PythonWithDrones*, einer vollständig browserbasierten Lernplattform, die darauf abzielt, grundlegende Python-Konzepte mithilfe interaktiver 3D-Simulationen zu vermitteln. In dieser Umgebung steuern Lernende eine virtuelle Drohne durch selbst geschriebene Python-Skripte. Die technische Umsetzung basiert auf mehreren zentralen Komponenten, welche im Verlauf der Arbeit detailliert beschrieben werden.

2 Einleitung

Die Vermittlung von Programmierkenntnissen stellt Lehrende und Lernende gleichermaßen vor grundlegende Herausforderungen. Abstrakte Konzepte wie Schleifen, Bedingungsanweisungen oder räumliches Denken sind für Einsteiger häufig schwer greifbar, da ein unmittelbares, visuelles Feedback fehlt. Traditionelle Lernumgebungen, in denen Code geschrieben und dessen Ausgabe ausschließlich in Textform erhalten wird, fördern zwar das syntaktische Verständnis, bieten jedoch wenig Motivation für Anfänger.

Unser Projekt *PythonWithDrones* soll dies adressieren, indem es Programmieren mit einem interaktiven, spielerischen Element verbindet. Lernende schreiben Python-Code, um eine virtuelle Drohne durch verschiedene Level zu steuern. Die Drohne muss dabei Hindernisse umfliegen und ein Zielportal erreichen. Der Drohnenzustand wird in Echtzeit in einer dreidimensionalen Visualisierung im Browser dargestellt, wodurch abstrakte Programmierkonzepte in eine konkret erfahrbare, räumliche Welt übertragen werden.

Die Grundidee des Projekts gründet in der Überzeugung, dass Programmieren für jeden erlernbar ist und ähnlich wie das Erlernen einer natürlichen Sprache durch Wiederholung und Übung gefestigt wird. Durch unmittelbares, visuelles Feedback und schrittweise steigende Schwierigkeitsgrade soll der Einstieg in die Programmierung ermöglicht und nachhaltig erleichtert werden. Das Projekt wurde im Rahmen des Hochschulmoduls *Software Design* entwickelt und demonstriert dabei sowohl innovative pädagogische Ansätze als auch bewährte moderne Softwarearchitekturprinzipien. Die vollständig browserbasierte Anwendung ist ohne jegliche lokale Installation oder Registrierung unter `https://pottmeier.github.io/PythonWithDrones/` erreichbar.

3 Grundlagen

Dieses Kapitel legt die konzeptionellen und technischen Grundlagen dar, auf denen *PythonWithDrones* aufbaut. Zunächst werden pädagogische Konzepte des spielbasierten Lernens erläutert, anschließend die eingesetzten Technologien beschrieben.

3.1 Spielbasiertes Lernen und Computational Thinking

Spielbasiertes Lernen nutzt Spielmechaniken gezielt, um Lernziele zu erreichen und die intrinsische Motivation der Lernenden zu steigern. Für das Erlernen von Programmiersprachen ist dieser Ansatz besonders geeignet. Durch sofortiges visuelles Feedback beobachten

Lernende die Auswirkungen ihres Codes direkt und können ein intuitives Verständnis von Schleifen, Bedingungen und Ablaufsteuerung aufbauen. Das Levelsystem von *PythonWithDrones* folgt dabei dem Prinzip des schrittweisen Aufbaus, bei dem Lernende sukzessive an komplexere Aufgaben herangeführt werden. Einfache Level erfordern lediglich grundlegende Bewegungsbefehle, während fortgeschrittene Level den Einsatz von Schleifen und Bedingungsanweisungen voraussetzen.

Darüber hinaus fördert das Projekt Fähigkeiten des strukturierten, algorithmischen Denkens, das für die moderne Informatik zentral ist. Das Navigieren einer Drohne durch einen Hindernisparcours erfordert die Zerlegung des Gesamtproblems in einzelne Teilschritte, das Erkennen von Bewegungsmustern sowie die präzise Formulierung eines Algorithmus in Python-Syntax. Diese Kompetenzen sind grundlegend für das Programmieren und bilden eine solide Basis für weiterführende Themen der Informatik und Softwareentwicklung.

3.2 Browserbasierte Python-Ausführung

Traditionell setzt die Ausführung von Python-Code eine lokale Interpreter-Installation voraus, was für Lernumgebungen eine erhebliche Einstiegshürde darstellt. Moderne Technologien ermöglichen es jedoch, [1] Python direkt im Browser auszuführen, ohne serverseitige Infrastruktur zu benötigen. Grundlage hierfür ist WebAssembly, ein binäres Instruktionsformat, das in modernen Browsern nahezu nativ ausgeführt werden kann. Auf dieser Basis ermöglicht [2] Pyodide die vollständige Ausführung des CPython-Interpreters im Browser.

PythonWithDrones nutzt diesen Ansatz konsequent. Die gesamte Anwendung ist als statische Webseite deployt und benötigt keinerlei Backend. Lernende benötigen ausschließlich einen modernen Browser, um sofort mit dem Programmieren zu beginnen. Wir haben diese Entscheidung getroffen um es allen Studierenden der Hochschule zu ermöglichen unser Projekt zu nutzen.

3.3 Verwendete Technologien

Die Implementierung basiert auf einem vollständig browserbasierten Technologie-Stack ohne serverseitige Infrastruktur. Tabelle 1 gibt einen Überblick über die eingesetzten Technologien, anschließend werden die Kernkomponenten näher erläutert.

3.3.1 Next.js und TypeScript

[3] Next.js ist ein auf React basierendes Framework, das dateibasiertes Routing, Static Site Generation sowie eine klare Projektstruktur mitbringt. Als typisierte Obermenge von JavaScript ermöglicht [4] TypeScript statische Typprüfung bereits zur Kompilierzeit, was besonders für die Definition von Datenstrukturen wie dem Drohnenzustand (`DroneState`) sowie der Schnittstelle zwischen Pyodide und der 3D-Engine von Vorteil ist. Die Entscheidung für Next.js ermöglicht es zudem, die Anwendung als vollständig statischen Export zu bauen und direkt über GitHub Pages auszuliefern, ohne einen eigenen Server betreiben zu müssen. Komponenten für Editor, 3D-Viewport und Level-Selektor sind als eigenständige React-Komponenten implementiert und können unabhängig voneinander entwickelt und

Tabelle 1: Technologie-Stack von *PythonWithDrones*

Technologie	**Kategorie**	**Funktion im System**
Next.js	Framework	UI-Routing, SSG, Projektstruktur
TypeScript	Sprache	Typsichere Implementierung, statische Typprüfung
Three.js	Bibliothek	3D-Visualisierung der Drohne und Level
Pyodide	Wasm-Runtime	Python-Ausführung im Browser
Python	Lernsprache	Drohnensteuerung via Befehls-API
CSS	Styling	Layout und visuelle Gestaltung
shadcn	UI-Framework	Fertige React-Komponenten für die Benutzeroberfläche
GitHub Actions	CI/CD	Automatisierter Build und Deployment
GitHub Pages	Hosting	Statisches, serverfreies Deployment

getestet werden.

3.3.2 Three.js

[5] *Three.js* abstrahiert die WebGL-API und bietet eine deklarative Schnittstelle zur Erstellung von 3D-Szenen im Browser. Konzepte wie Szene, Kamera, Beleuchtung und Geometrien entsprechen denen professioneller 3D-Engines und ermöglichen die räumliche Darstellung komplexer Welten. In *PythonWithDrones* wird *Three.js* eingesetzt, um Drohne, Hindernisse und Zielportal dreidimensional darzustellen. Die Drohnenbewegung wird über einen Zustandsvektor aus Position und Rotation gesteuert, den der Python-Interpreter bei jeder Befehlsausführung aktualisiert, sodass Lernende eine unmittelbare visuelle Rückmeldung auf ihren Code erhalten.

3.3.3 Pyodide

Pyodide kompiliert den CPython-Interpreter zu WebAssembly und ermöglicht damit die Ausführung von Python vollständig im Browser, ohne serverseitige Infrastruktur zu benötigen. In *PythonWithDrones* bildet Pyodide das architektonische Herzstück. Der von Lernenden geschriebene Code wird über die Pyodide-API ausgeführt, wobei Drohnenbefehle als Python-Funktionen wie etwa `move()`, `turn_left()` oder `turn_right()` in den Python-Namespace injiziert werden. Jeder Funktionsaufruf übersetzt sich in eine Zustandsänderung des Drohnenobjekts, die *Three.js* anschließend visualisiert. Diese Brücke zwischen Python und JavaScript bildet das Kernstück der gesamten Anwendung.

3.3.4 shadcn

[7] *shadcn* ist eine Sammlung von vorgefertigten, wiederverwendbaren React-Komponenten, die mit dem CSS-Framework Tailwind CSS gestaltet wurden. Sie ermöglichen eine schnelle und konsistente Entwicklung der Benutzeroberfläche, ohne sich um das Design und die manuelle Implementierung der einzelnen UI-Elemente kümmern zu müssen. In *PythonWithDrones* wurden shadcn-Komponenten für die professionelle Gestaltung von Buttons, Sidebars, Modals und anderen interaktiven UI-Elementen verwendet, um eine intuitive Bedienoberfläche zu schaffen.

3.3.5 GitHub Actions und Pages

Der gesamte Build- und Deploymentprozess wird über [8] *GitHub Actions* vollautomatisiert und kontinuierlich verwaltet. Bei jedem Push auf den `main`-Branch wird eine CI/CD-Pipeline ausgelöst, die den TypeScript-Code überprüft und kompiliert, den statischen Next.js-Export erzeugt und das Resultat auf [9] *GitHub Pages* veröffentlicht. Diese moderne DevOps-Kombination ermöglicht uns ein vollständig serverfreies, wartungsarmes und kostenloses Deployment, das Qualität und Verfügbarkeit sichert.

4 Implementierung

4.1 Systemarchitektur & Komponentenübersicht

In diesem Kapitel wird die Implementierung der vorgestellten Technologien beschrieben, welche im Projekt *PythonWithDrones* verwendet wurde. Die Architektur folgt dabei einem klaren Schichtenmodell (siehe Abbildung 1), das sich in vier Ebenen untergliedert: Die Präsentationsschicht mit React-Komponenten, die Business-Logic-Schicht für Zustandsverwaltung und Level-Generierung, die Python-Runtime in einem Web Worker für die Drohnenlogik, und schließlich die Datenzugriffsschicht für Speicherung und Level-Definitionen.

Der erste Bestandteil ist die React-Schicht, mit der die Nutzerinteraktionen verarbeitet und Code-Editor, Level-Auswahl, sowie Fortschrittsverwaltung bereitgestellt werden. Diese Präsentationsschicht ist mit der Business-Logic-Schicht über Custom Hooks wie `usePyodideWorker` verbunden. Der Spielfortschritt wird ohne Backend ausschließlich im localStorage des Browsers gespeichert, um eine serverseitige Infrastruktur zu vermeiden. Dazu gehören zum Beispiel die Info, welche Level geschafft wurden, oder der zuletzt geschrieben Code in einem Level. Das UI-Styling wird hier durch *shadcn* realisiert, wodurch fertige Komponenten für die Benutzeroberfläche, wie zum Beispiel Buttons oder die Sidebar, direkt in das Projekt integriert werden.

Die zweite wichtige Komponente ist die sichere Ausführung des Python-Codes mit Hilfe von Pyodide in der Python-Runtime-Schicht. Um die Anwendung nicht zu blockieren oder einzufrieren, wird der Code des Nutzers nicht im Hauptthread ausgeführt, sondern in einen Web Worker ausgelagert, der die Pyodide-Runtime sicher kapselt. So wird zuverlässig sichergestellt, dass React und *Three.js* ansprechbar bleiben, auch wenn der Nutzercode lange Schleifen oder rechenintensive Operationen enthält. Die Kommunikation zwischen

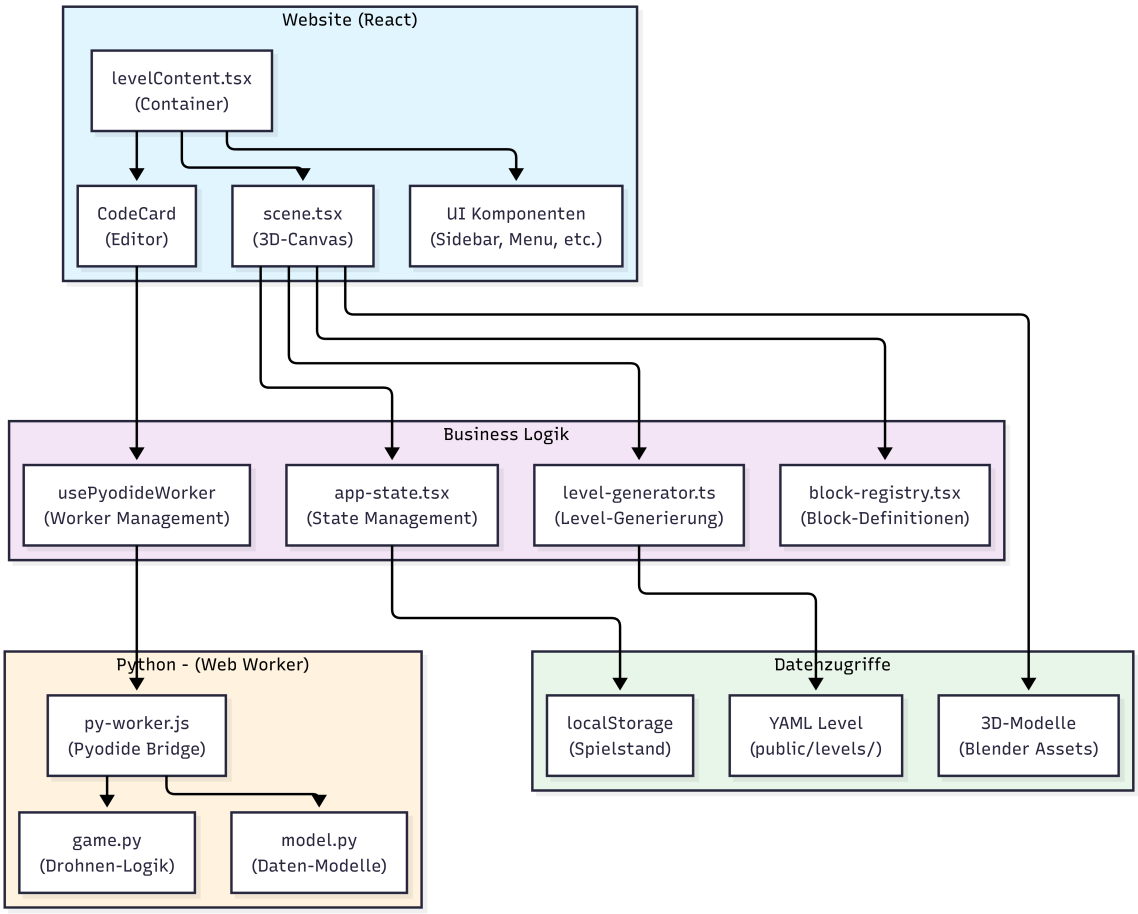


Abbildung 1: Schichtmodell der Systemarchitektur von *PythonWithDrones*. Die Präsentationsschicht (oben) verwaltet die Benutzerinteraktion, während die Business-Logic-Schicht diese mit der Python-Runtime verbindet. Die Python-Ausführung läuft isoliert in einem Web Worker ab, um die Bedienoberfläche responsiv zu halten. Die Datenzugriffsschicht (unten) verwaltet Level-Definitionen und Spielfortschritt.

dem Worker und Hauptthread erfolgt über das standardisierte und sichere `postMessage`-Protokoll. Dabei empfängt der Worker Befehle wie `LOAD_LEVEL` oder `RUN` und schickt dann strukturierte Nachrichten vom Typ `ACTION`, `FINISHED` oder `ERROR` zurück.

Außerdem gibt es noch die 3D-Darstellung der gesamten Szene in der Präsentationsschicht. Hier ist das globale `window`-Objekt die Brücke zwischen Python-Runtime und der angezeigten Visualisierung. Wenn der Worker eine `ACTION`-Nachricht sendet, wird durch den Hook `usePyodideWorker` die Funktion `window.droneAction` aufgerufen, die von der Szenenkomponente registriert wurde.

So kann jeder Befehl als strukturiertes Datenobjekt an die Animationsschicht übergeben werden, ohne dass Python und *Three.js* direkt voneinander abhängen. In der anderen Richtung werden über `window.getLevelData` und `window.getBlockRegistry` die für die Simulation benötigten Leveldaten und Blockinformationen aus der Datenzugriffsschicht bereitgestellt, auf welche der Worker beim Laden eines Levels zugreift.

In den folgenden Unterkapiteln werden die einzelnen Komponenten der Architektur detailliert beleuchtet. Das Level-System und die zugrunde liegende Datenstruktur, die Python-Ausführung in der isolierten Runtime, das verwendete Kommunikationsprotokoll zwischen den Schichten, und schließlich die 3D-Visualisierung mit *Three.js*.

4.2 Level-System & Datenstruktur

Jedes Level in *PythonWithDrones* wird in einer eigenen YAML-Datei definiert und beinhaltet alle relevanten Informationen: Id, Titel, Kurzbeschreibung, Tags, Beschreibung, Startpunkt der Drohne, Gewinnbedingung und die Levelgeometrie. Durch die Trennung von Levelinhalt und Anwendungslogik können leicht neue Level geschrieben und hinzugefügt werden, um das Projekt erweiterbar zu halten. Titel, Tags und Kurzbeschreibung werden für die Auswahl im Menü benötigt. Die Beschreibung wird im Level selber angezeigt und enthält eine kurze Problemstellung mit Lösungshinweisen, beispielsweise wie man Schleifen in Python verwendet.

Die Geometrie der Level ist als Voxelsystem aufgebaut, wodurch klare Regeln geschaffen werden, die leicht verständlich sind, damit der Nutzer sich auf die Programmierung mit Python fokussieren kann. In der YAML-Datei werden dafür beliebig viele Schichten (`layer_0`, `layer_1, etc.`) angelegt, welche jeweils einer horizontalen Ebene in der 3D-Welt entsprechen. Jede Ebene enthält dabei eine Matrix, in der die Blöcke über Block-IDs definiert sind. Durch dieses einfache Prinzip lassen sich beliebig komplexe 3D-Level erschaffen und in strukturierter Textform speichern.

Ein weiterer zentraler Bestandteil des Level-Systems ist die `block-registry.tsx`. In dieser Datei werden alle Arten von Blöcken mit ihren Eigenschaften definiert. So sind Informationen, wie die Block-ID, ob ein Block Kollisionen besitzt, ein Block als Ziel zählt, oder welche React-Komponente ihn visuell darstellt, festgelegt. Dadurch ergibt sich eine einzige Datenquelle, auf die sowohl von der virtuellen Python Logik, als auch von der visuellen 3D-Darstellung, zurückgegriffen wird. Die gesamte Anwendung bleibt so leicht wartbar und erweiterbar.

Beim Laden eines Levels wird dann durch `grid.tsx` die YAML-Datei eingelesen. Hier wird für jeden Block eine Instanz mit der zugehörigen React-Komponente aus der Registry erstellt. Zusätzlich werden über `window.getLevelData` die Daten des Levels für die virtuelle Python Berechnung bereitgestellt.

4.3 Python-Ausführung im Browser

Die Ausführung von Python-Code direkt im Browser ist das Schlüsselelement von *PythonWithDrones*. Grundlage dafür ist Pyodide, das den CPython-Interpreter via WebAssembly im Browser lauffähig macht. Ursprünglich wurde Pyodide direkt im Hauptthread betrieben, wodurch die Anwendung aber bei Endlosschleifen eingefroren ist. Als Lösung dafür

wird Pyodide deshalb in einen Web Worker ausgelagert, sodass auch eine `while True`-Schleife im Nutzercode vom Hauptthread isoliert wird und die Bedienoberfläche, sowie die 3D-Szene ansprechbar bleiben.

Der Worker wird beim Start eines Levels initialisiert, lädt dabei Pyodide zusammen mit den benötigten Python-Paketen (`pyyaml`, `pydantic`) und schreibt die `game.py` und `model.py` direkt in das virtuelle Dateisystem von Pyodide, damit diese zur Laufzeit importiert werden können. Die Kommunikation zwischen dem Hook `usePyodideWorker` und dem Worker wird über ein Nachrichtenprotokoll via `postMessage` realisiert. Dabei nutzt der der Worker drei eingehende Nachrichtentypen. `LOAD_LEVEL` wird zum Initialisieren eines neuen Levels benötigt. Zum Ausführen des Nutzercodes wird `RUN` genutzt. Außerdem gibt es den impliziten Initialisierungsschritt bei Start des Workers. Ausgehend werden die Nachrichtentypen `READY`, `ACTION`, `FINISHED` und `ERROR` verwendet, abhängig davon, ob eine Drohnenaktion ausgelöst wurde, die Ausführung erfolgreich war oder ein Fehler aufgetreten ist.

Der Nutzercode wird zudem nicht isoliert gestartet, sondern über `exec()` im Namespace von `game.py` ausgeführt. Auf diese Weise sind die definierten Objekte, wie die Drohne, im Nutzercode verfügbar, ohne zusätzliche Imports anzugeben. Lernende müssen sich so nicht mit den Modulstrukturen auseinandersetzen, sondern können sich auf das Wesentliche konzentrieren und Funktionen, wie `drone.move()` oder `drone.turn_left()`, direkt nutzen.

4.4 Kommunikation zwischen Python und JavaScript

Da Python im Web Worker und die 3D-Szene im Hauptthread laufen, können die beiden Seiten nicht direkt miteinander kommunizieren. Als Brücke dient die folgende Lösung. Mit jedem Drohnenbefehl wird in Python ein strukturiertes Datenobjekt, eine Action, erzeugt und mit Hilfe von `postMessage` an den Hauptthread gesendet. Diese Nachricht wird vom Hook `usePyodideWorker` entgegengenommen, welcher wiederum `window.droneAction` aufruft. Dadurch ist die virtuelle Logik in Python von der visuellen Darstellung durch *Three.js* entkoppelt. Es wird nur ein Datenobjekt gesendet, das die Bewegung auf ein bestimmtes Feld oder die Drehrichtung beschreibt.

Um eine saubere Animation der eingehenden Aktionen zu gewährleisten, werden diese nicht direkt in der 3D-Welt ausgeführt, sondern in einer Warteschlange eingereiht. Durch eine eigenständige Funktion wird dieser Puffer an Befehlen dann sequenziell abgearbeitet. Dabei muss immer eine Animation abgeschlossen sein, bevor die nächste ausgelöst wird. Dieses Prinzip stellt sicher, dass die Bewegungen in der richtigen Reihenfolge und ohne Überlappungen ausgeführt werden. Dadurch ist die visuelle Darstellung unabhängig von der Verarbeitungsgeschwindigkeit der virtuellen Drohne in Python, welche schneller abgeschlossen ist als die sichtbaren Animationen.

Eine konkrete Einschränkung entsteht jedoch bei Endlosschleifen, in denen die Drohne nicht abstürzt. Ohne eine Verzögerung würde Python in hoher Frequenz Befehle in die Warteschlange einreihen, was die gesamte Anwendung verlangsamen würde. Eine einfache Lösung wurde hier durch ein `time.sleep(0.1)` in der `__send_action__` umgesetzt. Durch diese Pause von 100 Millisekunden wird die Rate von eingehenden Befehlen auf ein kontrolliertes Maß begrenzt, ohne zusätzliche Logiken einzubauen.

Neben den regulären Aktionen gibt es bislang zwei Sonderfälle. Bei einem Absturz der Drohne sendet Python eine `crash`-Action, wodurch in der Szene eine Absturzanimation ausgelöst wird und verbleibende Aktionen in der Warteschlange verworfen werden. Gleichzeitig wird im Hook `usePyodideWorker` der Zustand `hasCrashed` auf `true` gesetzt, wodurch der Run-Button für den Nutzer deaktiviert bleibt, bis die Drone manuell zurückgesetzt wird. Beim Erreichen des Ziels sendet Python eine `goal`-Action, wodurch das Level in der Szene als abgeschlossen markiert und der Fortschritt im `localStorage` gespeichert wird.

4.5 Visualisierung der 3D-Szene

Die Szene wird mit Hilfe von *React Three Fiber* gerendert, einer deklarativen React-Abstraktion über *Three.js*. Sie erlaubt es die Szenebeleuchtung, Kamera und 3D-Objekte als gewöhnliche React-Komponenten zu behandeln.

Die Blockkomponenten nutzen derzeit zwei verschiedene Ansätze zur Darstellung. So wird das Zielportal noch mit einem einfachen geometrischen Primitive direkt im Code definiert. Andere Blöcke wurden bereits durch `.glb`-Dateien ersetzt, welche in *Blender* modelliert und in das Projekt geladen wurden. Die Drohne selbst ist als eigene Komponente implementiert und wird über eine `groupRef` angesteuert. Eingehende Aktionen aus der Warteschlange lösen dabei *GSAP*-Animationen für die Drohnenposition und -rotation aus. Die Geschwindigkeit der Drohne lässt sich über `gsap.globalTimeline.timeScale()` zur Laufzeit anpassen, damit Nutzer bei langen Programmabläufen die Animationen beschleunigen können, um so verschiedene Varianten zu testen, ohne zu lange warten zu müssen. Da es sich um eine dreidimensionale Spieltwelt handelt, ist die Kamera als frei drehbare Instanz konfiguriert, die Rotation, Translation und Zoom innerhalb der Levelgrenzen erlaubt.

5 Ergebnisse

PythonWithDrones liegt als funktionsfähiger Prototyp vor, welcher die Idee einer browserbasierten Lernplatform vollständig umsetzt. Die Anwendung umfasst bislang sechs spielbare Level, von denen jedes ein neues Konzept einführt. Dabei wird in Level 1 & 2 zunächst erklärt, wie man die Drohne in der Welt bewegen und drehen kann. In den folgenden Leveln werden dann grundlegende Programmierkonzepte wie Schleifen, Funktionen und bedingte Anweisungen vorgestellt. Die Level bauen dabei aufeinander auf und werden der Reihe nach freigeschaltet.

Lernenden steht hierfür eine minimale Python-API zur Verfügung. Es sind bereits Drohnenbefehle, wie `drone.move()` oder `drone.turn_left()`, sowie Zustandsabfragen, wie `is_path_blocked()` oder `at_portal()` verfügbar. Der Fortschritt und zuletzt geschrieben Code werden im Browser automatisch gespeichert, sodass die Anwendung jederzeit verlassen werden kann, um zu einem späteren Zeitpunkt fortzufahren. Außerdem wird keinerlei Installation benötigt, da das Projekt direkt über *GitHub Pages* unter `https://pottmeier.github.io/PythonWithDrones/` erreichbar ist und sowohl auf Desktop- und Mobilgeräten genutzt werden kann.

Das Hauptziel der Anwendung ist die Vermittlung von Programmierkonzepten durch in-

teraktive und spielerische Elemente. In *PythonWithDrones* wird jeder Python-Befehl in sichtbare Bewegungen der Drohne umgesetzt, wobei auch ein Fehler, wie das Abstürzen der Drohne mit einer entsprechenden Animation dargestellt werden. Da es als Lernplattform konzipiert ist, wird die Schwierigkeit mit jedem Level gesteigert.

6 Auswertung & Diskussion

Da es sich bei der Anwendung noch um einen Prototyp handelt, gibt es einige bekannte Einschränkungen. Zur Zeit wird der Web Worker noch bei jedem Zurücksetzen der Szene neu gestartet, statt nur einmalig beim Laden eines Levels initialisiert zu werden. Zudem ist noch nicht klar, wie die Anwendung von echten Nutzern angenommen wird, da bisher noch keine relevanten Tests mit Personen ohne Programmiererfahrung durchgeführt wurden. Es könnte sich herausstellen, dass die Lernkurve zu steil ist oder dass bestimmte Konzepte nicht ausreichend erklärt werden. Hier wäre es wichtig, Feedback von echten Nutzern einzuholen, um die Anwendung iterativ zu verbessern und an die Bedürfnisse der Zielgruppe anzupassen. Außerdem ist die Anzahl an Leveln noch begrenzt und dementsprechend noch nicht genug, um ein vollumfassendes Python-Tutorial darzustellen. Auf der anderen Seite, ist die Infrastruktur bereits soweit fertig, dass neue Level einfach hinzugefügt werden können, wodurch die Erweiterung des Lernangebots relativ unkompliziert möglich ist. Bereits jetzt können neue Level Requests über `https://github.com/pottmeier/PythonWithDrones/issues/new?template=level-request.md` eingereicht werden.

7 Schlussfolgerung

7.1 Zusammenfassung

PythonWithDrones demonstriert die Machbarkeit einer vollständig browserbasierten Lernplattform, die Python-Programmierung mit einem interaktiven, spielerischen Element verbindet. Durch die Kombination von *Pyodide* für die Python-Ausführung, *Three.js* für die 3D-Visualisierung und einem klar strukturierten Level-System bietet die Anwendung eine motivierende Umgebung für Einsteiger, um grundlegende Programmierkonzepte zu erlernen. Die Entscheidung für ein serverfreies Deployment über *GitHub Pages* ermöglicht einen einfachen Zugang für alle Nutzer, ohne technische Hürden. Die Anwendung befindet sich im Bezug auf die Level noch in einem frühen Stadium, bietet aber bereits eine solide Grundlage mit der nötigen Infrastruktur, um das Lernangebot sukzessive zu erweitern.

7.2 Nächste Schritte und Erweiterungsmöglichkeiten

- **Benutzertests:** Durchführung von Tests mit Personen ohne Programmiererfahrung, um die Benutzerfreundlichkeit und die Effektivität der Lerninhalte zu evaluieren.
- **Level:** Erweiterung des Level-Angebots, um ein umfassenderes Python-Tutorial zu bieten.
- **Level-Editor:** Einführung eines integrierten Editors, der es ermöglicht, neue Level einfach zu erstellen und anzupassen.
- **Randomisierter Level-Generator:** Entwicklung eines Systems, das automatisch variierende Level erzeugt, um für mehr Abwechslung und langfristige Motivation zu sorgen.
- **Optimierung der Web-Worker-Logik:** Verbesserung der Performance, indem der Web Worker nur einmal beim Laden eines Levels initialisiert wird und nicht bei jedem Reset der Drohne.
- **Geräteübergreifende Speicherung:** Implementierung einer Funktion, die es Nutzern erlaubt, ihren Fortschritt über verschiedene Geräte hinweg zu speichern und abzurufen.
- **Leaderboard:** Einführung einer Rangliste, die Nutzer dazu motiviert, Level effizient zu lösen und sich mit anderen zu messen.

8 Literaturverzeichnis

Literatur

[1] Python Documentation https://docs.python.org

[2] Pyodide Documentation https://pyodide.org

[3] Next.js Documentation https://nextjs.org

[4] TypeScript Documentation https://www.typescriptlang.org

[5] Three.js Documentation https://threejs.org

[6] CSS Documentation https://developer.mozilla.org/en-US/docs/Web/CSS

[7] shadcn Documentation https://ui.shadcn.com/docs

[8] GitHub Actions Documentation https://docs.github.com/en/actions

[9] GitHub Pages Documentation https://docs.github.com/en/pages